NEW HORIZON
COLLEGE OF ENGINEERING

Department of Artificial Intelligence & Machine Learning

Academic Year 2025–26 (EVEN)

Report

for

Mini Project - I (24AIM48)

on

FRISBEE

Functional Realtime Intelligence for Spotting Blissful Everyday Actions

A personal logistic regression model trained on daily free-form text to predict individual weekly behavioural alignment.

By

Bhavishya Srivastava	1NH24AI197
Arya Navaneeth	1NH24AI217
AJL Hansika	1NH24AI216
Vunnam Manogna	1NH24AI211

Under the Guidance Of

Prof. Shashikala K S

Senior Assistant Professor
Dept. of AI & ML, NHCE, Bangalore, KA, India

Certificate

This is to certify that the project **FRISBEE: Functional Realtime Intelligence for Spotting Blissful Everyday Actions** was carried out by Bhavishya Srivastava (1NH24AI197), Arya Navaneeth (1NH24AI217), AJL Hansika (1NH24AI216), and Vunnam Manogna (1NH24AI211) as part of Mini Project – I (24AIM48) in the Department of Artificial Intelligence and Machine Learning, New Horizon College of Engineering, Bangalore, during the academic year 2025–26. The work is original and was completed under the guidance of the undersigned.

Prof. Shashikala K S
Project Guide
Dept. of AI & ML, NHCE

Dr. N V Uma Reddy
Head of Department
Dept. of AI & ML, NHCE

External Examiner

Acknowledgement

The satisfaction and euphoria of completing any task are impossible without the guidance and encouragement of those who made it possible.

We thank **Dr. Mohan Manghnani**, Chairman, New Horizon Educational Institutions, for the infrastructure and environment that made this work possible.

We thank **Dr. Manjunatha**, Principal, and **Dr. R. J. Anandhi**, Dean Academics, for their constant support and encouragement.

We thank **Dr. N. V. Uma Reddy** and the Department of Artificial Intelligence and Machine Learning for an environment that takes project-based learning seriously.

We thank **Prof. Shashikala K S** for guiding this project. Her questions consistently pushed us toward greater precision, and her patience gave us room to build something we genuinely care about.

Bhavishya Srivastava
Arya Navaneeth
AJL Hansika
Vunnam Manogna

Abstract

People routinely overestimate how well they will behave in the coming week. This is not random error. It is a consistent, measurable bias rooted in how humans construct self-predictions: drawing on idealised self-concept rather than behavioural history. No existing tool attempts to learn this gap from a single person's own words.

FRISBEE is a browser-based system in which a logistic regression model, trained exclusively on one user's daily free-form messages, predicts whether each week will be one of behavioural alignment or not. Each message is converted into a four-dimensional feature vector via cluster-weighted TF-IDF scoring across four semantic dimensions: *drift*, *align*, *excuse*, and *energy*. A blended inverse document frequency formula, combining personal and background corpus statistics, resolves the cold-start problem in the early weeks of use. At the end of each week, the model's prediction is compared against the user's own blinded self-prediction in a direct duel. Over time the model learns the user's specific language patterns, and by extension, their specific optimism bias.

All computation runs on the user's device. No data is transmitted. The system is implemented as a zero-dependency TypeScript ML pipeline and deployed as a live web application.

Keywords: logistic regression, TF-IDF, behavioural prediction, optimism bias, on-device machine learning

Contents

List of Figures

List of Tables

Chapter 1. Introduction

1.1 Background

There is a consistent gap between how people predict their own behaviour and how they actually behave. Research in cognitive psychology and decision science has established that self-predictions draw on idealised self-concept rather than behavioural history, producing estimates that are systematically biased. Sharot [9] identified this as the optimism bias: a universal tendency to overestimate the probability of positive future outcomes. Dunning et al. [10] demonstrated that the failure is not simply a lack of information. Even when people have direct access to their own past behaviour, they do not consult it when forming predictions about themselves. The estimate is constructed from how a person thinks about themselves, not from evidence of how they have acted.

This pattern is most visible in adolescents. The teenage years involve rapid change in identity and self-concept, and the distance between aspiration and action is felt acutely. The everyday language of this experience is precise: *I said I would study and did not. Why do I keep doing this.* These are not productivity complaints. They are observations about a gap between who a person imagines themselves to be and who actually shows up. FRISBEE takes this gap seriously as a research problem and asks whether it is learnable from the person's own words.

1.2 Problem Statement

Self-prediction of weekly behaviour fails in a specific and exploitable way. The prediction is formed from self-concept, not from evidence. The language a person uses each day, however, is closer to evidence. Words like *procrastinated*, *exhausted*, *finally finished*, and *pumped* are not arbitrary choices. They reflect the actual state of the day as the person experienced it, written close to the time of the experience.

The central question this project addresses is: given a person's daily free-form messages over several weeks, can a machine learning model trained exclusively on those messages predict whether any given week was one of behavioural alignment more accurately than the person can predict it themselves?

1.3 Objectives

1. Design a text feature extraction pipeline that converts free-form daily messages into a four-dimensional feature vector via cluster-weighted TF-IDF with a blended inverse document frequency formula.
2. Build a logistic regression model with L2 regularisation that trains incrementally on one user's labelled weekly data, with no external training corpus and no pre-trained weights.
3. Implement a weekly prediction duel in which the model's prediction is compared against the user's own blinded self-prediction, with outcome scores tracked over time.

4. Implement a hypothesis surfacing module that identifies words statistically associated with the user's good or bad weeks using Bernoulli confidence estimation.
5. Store all data locally on the user's device using IndexedDB, with no server component and no data transmission.
6. Deploy the system as a functional minimum viable product and demonstrate the viability of individual-level behavioural alignment prediction from natural language.

1.4 Literature Survey

1.4.1 Sweller [1] — Cognitive Load Theory

Sweller established that when a person's working memory is occupied, the quality of secondary cognitive tasks degrades significantly. This finding motivates FRISBEE's choice of free-form text input over structured tracking forms. Dashboards and checklists impose a cognitive load through their structure. A single open-ended daily message imposes almost none. The system extracts structure from unstructured input rather than demanding structure from the user.

1.4.2 Kahneman [2] — Thinking, Fast and Slow

Kahneman describes two modes of cognition. System 1 is fast, automatic, and operates without deliberate effort. System 2 is slow and effortful. Self-prediction tends to rely on System 1: a person generates a weekly prediction based on how they feel about themselves in that moment. The logistic regression model in FRISBEE functions as an external System 2 proxy. It reviews the actual words written over the week and produces a prediction based on observed patterns, not on current mood.

1.4.3 Rebetez et al. [3] — Procrastination and Self-Regulation

Rebetez et al. identify three correlated factors in procrastination behaviour: cognitive (poor planning), emotional (difficulty regulating negative affect), and motivational (low outcome expectancy). These factors manifest in language. Words such as *supposed to*, *tired*, and *tomorrow* are markers of emotional avoidance and motivational failure. The *excuse* cluster in FRISBEE's vocabulary was designed to capture exactly this class of signal.

1.4.4 Anderson [4] — Decision Avoidance

Anderson provides a taxonomy of decision avoidance behaviours including deferral and omission. Adolescents show elevated rates of decision avoidance when outcomes are uncertain or when they anticipate negative emotional consequences. FRISBEE's weekly commit mechanism addresses this by creating a single, low-friction binary decision point at the end of each week: was this a yes-week or a no-week?

1.4.5 Lee and See [5] — Trust in Automation

Lee and See identify two failure modes in human interaction with automated systems. Over-reliance occurs when users follow automation regardless of accuracy. Under-reliance occurs when users dismiss useful automated output. Both degrade outcomes. The prediction duel in FRISBEE is designed to address this: the user makes their own prediction before seeing the model's, preventing anchoring, and the duel score over time provides

calibration evidence without requiring the user to interpret model internals.

1.4.6 Hengstler et al. [6] — Applied AI and Trust

Hengstler et al. argue that trust in applied AI systems must be earned through demonstrated reliability, not claimed upfront. FRISBEE is designed with this in mind. The model is initialised with zero weights and makes near-random predictions in its first few weeks. The duel score makes this visible, allowing trust to be calibrated to actual performance.

1.4.7 Sparck Jones [7] — Inverse Document Frequency

Sparck Jones introduced inverse document frequency as a measure of term specificity. A term appearing in many documents provides less information than one appearing in few. This is the foundation of the IDF component in FRISBEE's feature extractor. Words a user writes every day carry no alignment signal. A word they rarely use, but wrote today, carries significant information about what kind of day it was.

1.4.8 Salton and Buckley [8] — Term-Weighting in Retrieval

Salton and Buckley evaluated term-weighting schemes systematically, establishing TF-IDF as a computationally efficient and reliable method for encoding document content. Their analysis confirms that combining local term frequency with global term specificity produces more informative representations than either component alone, which FRISBEE applies directly.

1.4.9 Sharot [9] — The Optimism Bias

Sharot established the optimism bias as one of the most consistent findings in cognitive science: people across cultures systematically overestimate positive future outcomes. The bias is structural, tied to how the brain constructs self-predictions. FRISBEE operationalises this finding. The gap between a user's self-prediction and the model's prediction is, in aggregate, a measure of their personal optimism bias magnitude.

1.4.10 Dunning et al. [10] — Flawed Self-Assessment

Dunning, Heath, and Suls review evidence that people are systematically poor judges of their own behaviour, even with access to their own history. Self-assessments are constructed rather than retrieved: people generate an estimate from self-concept rather than consulting their record. FRISBEE bypasses this construction process. The model is trained on the actual behavioural record, not on the user's narrative about it.

1.5 Existing Systems

Table 1.1 surveys the tools most relevant to FRISBEE's problem space and identifies where each falls short.

Table 1.1. Comparison of existing approaches

System type	Approach	Limitation
Habit trackers	Binary daily completion counts	No language analysis, no prediction, no model of the intention-to-action gap.
Journaling apps	Free-form text storage for recall	No modelling. Any "insight" relies on population-trained sentiment classifiers applied to the individual.
Sentiment APIs	Pre-trained affect classification	Models trained on crowd data. Individual patterns are averaged away. Not predictive of future outcomes.
Goal dashboards	Structured progress tracking	Require structured input. Do not learn from language and do not predict future alignment.

The common failure is that all existing approaches are either built for populations or require structured input. None learns from a single user's unstructured language, and none models the intention-to-action gap at the individual level.

1.6 Proposed System

FRISBEE is a browser-based application in which a logistic regression model is trained exclusively on one user's daily free-form messages. The model converts each message into a feature vector using four hand-designed semantic clusters and blended TF-IDF scoring, then predicts weekly behavioural alignment as a binary outcome. At the end of each week the user commits a self-prediction, the model reveals its own, and the user provides the ground truth label. The model then trains on the week's messages and updates its weights accordingly.

All computation and storage occur on the user's device. No data is transmitted. The system becomes more accurate as the user's personal corpus accumulates, learning the specific words and patterns that predict alignment for that person and no one else.

Chapter 2. System Requirements

2.1 Hardware Requirements

FRISBEE is a client-side web application. There are no server-side hardware requirements. Table 2.1 lists the requirements for the device running the application.

Table 2.1. Hardware requirements

Component	Requirement
Processor	Any processor capable of running a modern web browser. No specific instruction set is required.
Memory	Minimum 2 GB RAM. 4 GB is recommended for comfortable browser performance alongside other applications.
Storage	Minimum 50 MB available for IndexedDB. Typical usage over one year of daily messages is under 5 MB.
Display	Any screen capable of rendering a modern browser. The application targets a mobile-first viewport of approximately 390 px width.
Network	An internet connection is required only for the initial application load. All subsequent computation and storage are fully offline.

2.2 Software Requirements

Table 2.2 lists the software components required for development and deployment.

Table 2.2. Software requirements

Component	Requirement
Web browser	Any browser supporting IndexedDB, the Web Audio API, and ES2022 JavaScript. Tested on Chrome 120+, Firefox 121+, and Safari 17+.
Dev runtime	Bun 1.x or Node.js 20+ for local development and build. Not required by end users.
Build tool	Vite 7, included in project dependencies.
Language	TypeScript 5.8, compiled to ES modules. No TypeScript runtime dependency in production.
Deployment	Any static file host or edge worker runtime. No server-side runtime required.

2.3 Key Dependencies

The application uses no external machine learning library. The complete ML pipeline is implemented in plain TypeScript. The following third-party libraries handle non-ML re-

sponsibilities.

- **idb** — Promise-based IndexedDB wrapper. Used exclusively in `vault.ts` for data persistence.
- **Zustand** — Lightweight React state management. Used in `brain.ts` for the mascot state machine.
- **TanStack Router** — File-based routing for the five application routes.
- **React 19** — UI rendering. No React-specific data processing.
- **Tailwind CSS v4** — Utility-class styling with no effect on application logic.

Chapter 3. System Design

3.1 Architecture

The system is split into two independent modules that share no direct imports, as shown in Figure 3.1. The prediction pipeline (`src/lib/frisbee/`) handles all machine learning and data management. The UX layer (`src/lib/buddy/`) handles feedback to the user: mascot behaviour, sound, and haptics. The two modules communicate only through a typed event bus. This separation means the ML pipeline can be reasoned about and tested entirely independently of the interface layer.

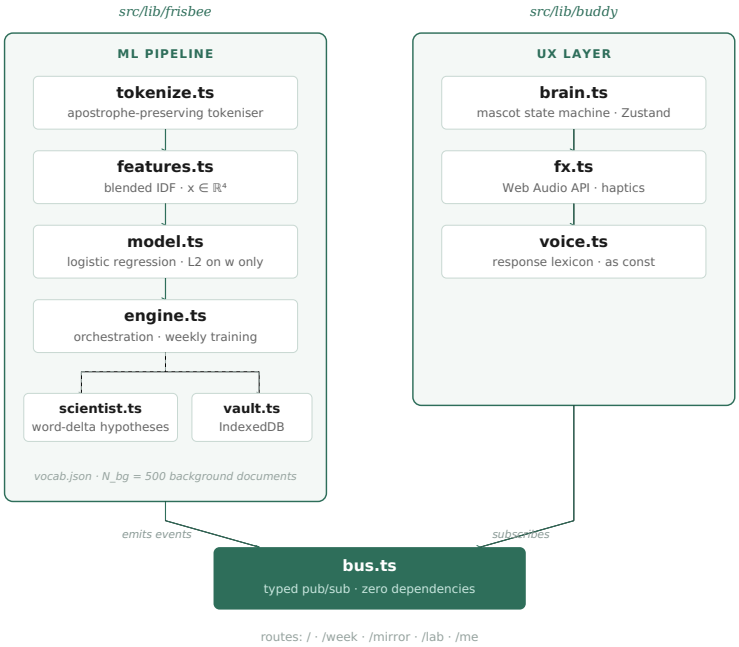

Figure 3.1. System architecture. The two modules communicate only through the typed event bus.

3.2 Tokenisation

The tokeniser in `tokenize.ts` converts raw text to a list of lowercase tokens while preserving apostrophes inside words. The design criterion is the preservation of contractions. The word *didn't* belongs to the *excuse* cluster. A naive tokeniser that strips punctuation splits it into *didn* and *t*, both of which match nothing. The signal is lost.

The tokeniser applies three transformations in order. First, the input is lowercased. Second, Unicode apostrophe variants (U+2018, U+2019, U+02BC) are replaced with the standard ASCII apostrophe, handling mobile keyboard input. Third, the regular expression

```
/[a-z0-9][a-z0-9'']*[a-z0-9] | [a-z0-9]/g
```
(3.1)

extracts tokens. This pattern permits apostrophes only in the interior of a token, not at

its boundaries, so contractions are preserved while surrounding punctuation is discarded correctly.

3.3 Feature Extraction

3.3.1 Cluster Vocabulary

The vocabulary in `vocab.json` defines four semantic clusters. Table 3.1 lists representative words from each.

Table 3.1. Semantic clusters with representative words

Cluster	Words (sample)
drift	procrastinated, scrolled, wasted, avoided, snoozed, skipped, forgot, didn't, nothing, again
align	focused, finished, shipped, studied, built, completed, proud, actually, finally, crushed
excuse	tired, exhausted, tomorrow, later, busy, overwhelmed, but, almost, should, couldn't
energy	motivated, pumped, inspired, excited, happy, great, lit, grateful, let's, locked

Words may belong to more than one cluster. The words *couldn't*, *didn't*, and *wouldn't* appear in both *drift* and *excuse*. The words *locked* and *crushing* appear in both *align* and *energy*. Such words contribute their TF-IDF score to every cluster they belong to.

3.3.2 TF-IDF Computation

For each token w in a submitted message, the term frequency is:

$$\mathrm{TF}(w) = \frac{\mathrm{count}(w)}{\text{total tokens}} \tag{3.2}$$

The personal IDF, given the user's accumulated corpus of N_p messages with document frequency $df_p(w)$, is:

$$\mathrm{IDF}_p(w) = \log\left(\frac{N_p + 1}{df_p(w) + 1}\right) + 1 \tag{3.3}$$

The background IDF is pre-computed over a corpus of $N_{bg} = 500$ documents stored in `vocab.json`. For words absent from the background corpus, $df_{bg}(w) = 1$ (not zero):

$$\mathrm{IDF}_{bg}(w) = \log\left(\frac{N_{bg} + 1}{df_{bg}(w) + 1}\right) + 1 \tag{3.4}$$

The blended IDF is a weighted average, with weights proportional to the respective corpus sizes:

$$\mathrm{IDF}_{blend}(w) = \frac{N_p \cdot \mathrm{IDF}_p(w) + N_{bg} \cdot \mathrm{IDF}_{bg}(w)}{N_p + N_{bg}} \tag{3.5}$$

When $N_p = 0$ (no messages yet), equation (3.5) reduces to $\mathrm{IDF}_{bg}(w)$, providing non-zero feature values from day one. As N_p grows, the personal IDF progressively dominates the blend. This resolves the cold-start problem without requiring any minimum usage period.

The score for cluster c is the sum of TF-IDF weights over all tokens in the message that belong to that cluster:

$$x_c = \sum_{w \in \text{msg} \cap C_c} \text{TF}(w) \cdot \text{IDF}_{blend}(w) \tag{3.6}$$

The full feature vector is:

$$\mathbf{x} = \left[x_{\text{drift}},\ x_{\text{align}},\ x_{\text{excuse}},\ x_{\text{energy}}\right] \in \mathbb{R}^4 \tag{3.7}$$

3.4 The Logistic Regression Model

3.4.1 Forward Pass

The model maintains a weight vector $\mathbf{w} \in \mathbb{R}^4$ and a scalar bias $b \in \mathbb{R}$, both initialised to zero. The linear combination is:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{4} w_i\, x_i + b \tag{3.8}$$

The value z is clipped to $[-500,\ 500]$ before the sigmoid to prevent floating-point overflow. The prediction is:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \hat{p} \in (0, 1) \tag{3.9}$$

A binary prediction is obtained by thresholding at 0.5. The value $\hat{p}$ is the model's estimate of the probability that the current week is a yes-week.

3.4.2 Loss and Parameter Update

After the user provides the ground truth label $y \in \{0, 1\}$ at the end of the week, the loss is binary cross-entropy:

$$\mathcal{L} = -\left[y \log \hat{p} + (1 - y) \log(1 - \hat{p})\right] \tag{3.10}$$

The gradient of $\mathcal{L}$ with respect to $\mathbf{w}$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (\hat{p} - y)\, \mathbf{x} \tag{3.11}$$

This is the defining result of logistic regression: the gradient is the prediction error times the input. With L2 regularisation on the weights, the update rule for each weight is:

$$w_i \ \leftarrow\ w_i - \eta\left[(\hat{p} - y)\, x_i + \lambda\, w_i\right], \quad i = 1, 2, 3, 4 \tag{3.12}$$

The bias is updated without regularisation:

$$b \ \leftarrow\ b - \eta\,(\hat{p} - y) \tag{3.13}$$

The hyperparameters are fixed at $\eta = 0.1$ (learning rate) and $\lambda = 0.01$ (L2 strength). The bias is excluded from regularisation deliberately. Penalising b would pull predictions toward $\hat{p} = 0.5$ regardless of the evidence. The bias encodes the user's personal base rate of yes-weeks, which is genuine signal and must not be suppressed.

The engine applies one gradient step per daily message in the week, running them in chronological order. If the user submitted messages on five days of the week, the model receives five updates in that training epoch.

3.5 Hypothesis Surfacing

The scientist module (`scientist.ts`) operates as a side process that does not affect model predictions. It computes a word-delta analysis across all labelled messages. For each word w in the labelled history, it computes the mean term frequency on yes-week messages ($\bar{t}_{y=1}$) and on no-week messages ($\bar{t}_{y=0}$). The delta is:

$$\delta(w) = \bar{t}_{y=1}(w) - \bar{t}_{y=0}(w) \tag{3.14}$$

Words are ranked by $|\delta(w)|$. Each candidate is tracked as a hypothesis with a Bernoulli confidence score:

$$\text{confidence}(h) = \frac{k_h}{n_h} \tag{3.15}$$

where n_h is the number of weeks in which the word appeared and k_h is the number of those weeks in which the outcome matched the predicted direction. A hypothesis is surfaced to the user only when $n_h \geq 14$ and confidence ≥ 0.75. It is discarded when $n_h \geq 14$ and confidence < 0.25. These thresholds prevent the system from surfacing hypotheses before it has enough evidence to be credible.

3.6 Database Schema

All data is stored in a single IndexedDB database named `frisbee-vault` at schema version 1. Table 3.2 describes each object store.

Table 3.2. IndexedDB object stores

Store	Key	Contents
messages	Auto-increment	Timestamp, text, feature vector $\mathbf{x}$, prediction $\hat{p}$, week start. Indexed by week start and timestamp.
weeks	Week start	Self-prediction, model prediction, ground truth y, revealed flag.
model	`"current"`	Weight vector $\mathbf{w}$, bias b, update count, last updated timestamp.
corpus	`"personal"`	Total message count N_p, document frequency map df_p.
vocab	Auto-increment	User-added vocabulary entries with cluster and timestamp.
hypotheses	UUID string	Word, direction, n_h, k_h, created timestamp.

3.7 Data Flow

Figure 3.2 shows how a single daily message moves through the prediction pipeline from input to storage.

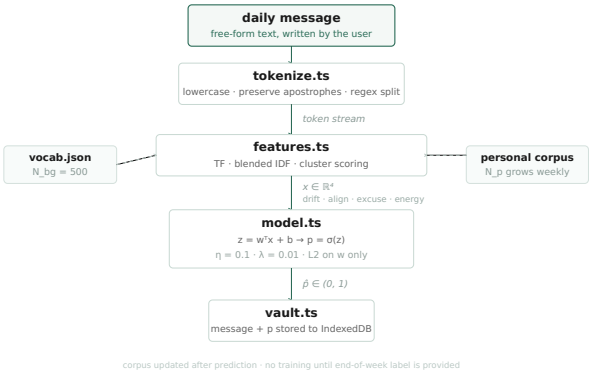

Figure 3.2. Data flow for a single daily message submission.

Chapter 4. Implementation

4.1 Module Overview

Table 4.1 lists every implementation file with its module and responsibility.

Table 4.1. Implementation modules

File	Module	Responsibility
`types.ts`	frisbee	Type definitions shared across the ML pipeline.
`tokenize.ts`	frisbee	Apostrophe-preserving tokeniser.
`features.ts`	frisbee	Blended IDF feature extractor.
`model.ts`	frisbee	Logistic regression: predict, loss, gradient step.
`scientist.ts`	frisbee	Word-delta hypothesis surfacing.
`vault.ts`	frisbee	IndexedDB persistence layer.
`engine.ts`	frisbee	Orchestration: message submission and weekly training.
`vocab.json`	frisbee	Hand-curated cluster words and background corpus.
`bus.ts`	buddy	Typed pub/sub event bus.
`brain.ts`	buddy	Mascot state machine (Zustand).
`fx.ts`	buddy	Web Audio API and haptics layer.
`voice.ts`	buddy	Response lexicon and selection.

4.2 The ML Pipeline

4.2.1 engine.ts

The engine is the only entry point from the UI into the ML pipeline. It exposes two asynchronous functions. `submitMessage(text)` tokenises the input, extracts the feature vector, generates the prediction, stores the message to the vault, and updates the personal corpus. The corpus update uses only the unique tokens from the submitted message: each unique word increments df_p by one. Crucially, the corpus update occurs after prediction, so the current message does not influence its own IDF weights.

`trainOnWeek(week)` retrieves all messages for the given week start timestamp and runs one gradient step per message in chronological order. The model state is written back to IndexedDB after the final step of the epoch.

The function `weekStartOf(ts)` computes Monday 00:00 local time for any given Unix timestamp. This value serves as the primary key for week records, ensuring messages from the same calendar week are consistently grouped regardless of when they were submitted.

4.2.2 model.ts

The gradient step is the core computation of the system. Algorithm 1 presents the procedure.

Algorithm 1 Single gradient step

Require: model $(\mathbf{w}, b)$, feature vector $\mathbf{x}$, truth $y \in \{0, 1\}$, $\eta = 0.1$, $\lambda = 0.01$

$z \leftarrow \text{clip}(\mathbf{w}^\top \mathbf{x} + b,\ -500,\ 500)$
$\hat{p} \leftarrow 1/(1 + e^{-z})$
$\text{err} \leftarrow \hat{p} - y$
for $i = 1$ to 4 **do**
    $w_i \leftarrow w_i - \eta\, \text{err}\, x_i - \eta\, \lambda\, w_i$ ▷ L2 on weights only
end for
$b \leftarrow b - \eta\, \text{err}$ ▷ no regularisation on bias
return updated $(\mathbf{w}, b)$

The step function does not mutate its input. It returns a new model state object and increments the update counter. This makes the training history traceable from the stored state.

A Python reference implementation (`collect.py`) applies the same formulae, and a parity test enforces byte-identical feature vectors for identical inputs across both languages. This confirms that the mathematical specification in Chapter 3 is implemented without approximation.

4.3 The Event Bus

`bus.ts` is a zero-dependency publish/subscribe module. It maintains a `Map` from event type to a `Set` of handler functions. The full event type union is a TypeScript discriminated union, giving each subscriber complete type narrowing when it inspects the event type before accessing payload fields. This is why no subscriber in the codebase requires a runtime cast.

The disposer returned by `on()` re-resolves the set from the map at call time rather than closing over the local reference at registration time:

```
return () => subs.get(type)?.delete(fn);
```

This ensures that if the underlying `Set` were ever replaced in a refactor, the disposer would still work correctly rather than silently becoming a no-op and leaking subscriptions.

4.4 The Buddy Layer

4.4.1 brain.ts

The mascot brain is a Zustand state machine with eight moods. Moods decay back to *neutral* after a configurable time-to-live via a module-level timer. The function `dominantCluster` maps the feature vector $\mathbf{x}$ to a mascot reaction by selecting the cluster with the highest score. This is the only point at which ML output influences UX behaviour; the mascot never reads model weights or probabilities directly.

4.4.2 fx.ts

The sound layer generates audio via the Web Audio API using no audio files. Each event produces one or more oscillator nodes with amplitude envelopes tuned for the event type. Haptic feedback is provided via `navigator.vibrate` where available. The audio context is created only after the first user gesture, respecting browser autoplay restrictions.

4.4.3 voice.ts

The response lexicon is compiled with the TypeScript `as const` assertion, which narrows every string array to its exact literal types. Without this, the `pick()` function returns `string` and the compiler cannot catch missing or misspelled keys at build time. The `pick()` function accepts an optional integer seed for deterministic selection, useful for preventing response flicker on re-render.

4.5 Application Routes

The application has five routes, shown in Figure 4.1.

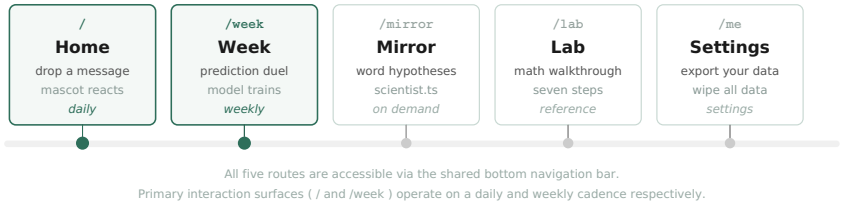

Figure 4.1. Application routes and shared bottom navigation bar.

4.6 The Weekly Cycle

Figure 4.2 shows the complete prediction and training loop that the user moves through each week.

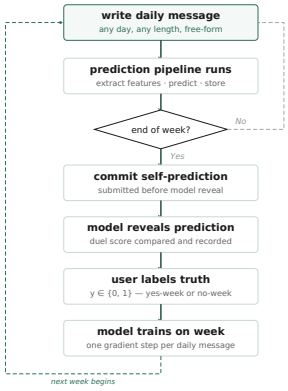

Figure 4.2. Weekly prediction and training cycle.

Chapter 5. Results and Discussion

5.1 MVP Status

The current implementation is a fully functional minimum viable product, deployed at `myfrisbee.lovable.app`. All modules described in Chapters 3 and 4 are operational. The system correctly tokenises daily messages, extracts feature vectors, generates predictions, stores data to IndexedDB, and updates model weights after each weekly training event. The prediction duel is live: the user submits a self-prediction, the model reveals its own, and the score is tracked across weeks.

No multi-week longitudinal data was collected during the development period. The results in this chapter are therefore analytical. They describe the expected behaviour of the system based on the mathematics of the model and the structure of the implementation.

5.2 Expected Performance

5.2.1 Cold Start

In the first few weeks, N_p is small and the blended IDF is dominated by the background corpus. The weights $\mathbf{w}$ are initialised to zero, so $\hat{p} = \sigma(0) = 0.5$ on the first message. Both the model and the user are making near-chance predictions at this stage, so the duel score will tend toward *both wrong* or *both right* roughly equally.

5.2.2 Convergence

As N_p grows, the personal IDF progressively dominates the blend and the feature vectors become more discriminative. The model will begin to assign consistent weight to words that reliably co-occur with yes-weeks or no-weeks for this specific user. The magnitude of weight updates, $\|\Delta\mathbf{w}\|$, will rise initially and then stabilise as the weights converge. Table 5.1 summarises the expected behaviour by phase.

Table 5.1. Expected model behaviour by phase

Phase	Weeks	Expected behaviour
Cold start	1–3	Background IDF dominates. $\hat{p} \approx 0.5$. Both model and user at near chance.
Learning	4–10	Personal IDF asserts. Weights shift from zero. Predictions grow more confident.
Convergence	10+	Weights stabilise. Model MAE expected to fall below user self-prediction MAE.

5.2.3 Primary Metric: MAE

The primary evaluation metric is mean absolute error between the binary prediction and the ground truth over a held-out test period. For the model and the user respectively:

$$\text{MAE}_{\text{model}} = \frac{1}{T}\sum_{t=1}^{T} |\hat{p}_t^{\text{rounded}} - y_t| \qquad \text{MAE}_{\text{user}} = \frac{1}{T}\sum_{t=1}^{T} |s_t - y_t| \tag{5.1}$$

where s_t is the user's self-prediction and y_t is the ground truth for week t. A four-week held-out period ($T = 4$) is planned. This is a proof-of-concept scale: the sample is too small for significance testing, but it is sufficient to observe the direction of the comparison.

5.3 Limitations

1. **No longitudinal data.** The system has not been evaluated over a complete multi-week period with real user data. All expected performance figures are analytical.
2. **Fixed vocabulary.** The cluster word lists are hand-curated. A user whose language falls outside these lists will produce weak feature vectors regardless of model quality. The user can add words via `/me`, but the base coverage is fixed at build time.
3. **Self-report bias.** The label y is user-provided at week end. Mood at labelling time can distort the signal: a difficult Sunday may retroactively make a neutral week feel like a no-week.
4. **Binary outcome.** Modelling alignment as a binary variable discards gradations. A week that was *almost* a yes-week and a clearly failed week receive identical labels.
5. **No confidence display.** The application shows the model's binary prediction but not $\hat{p}$. Per Hengstler et al. [6], showing calibrated confidence supports adaptive trust calibration. This is currently absent.

Chapter 6. Conclusion

6.1 Summary

FRISBEE demonstrates that a logistic regression model trained exclusively on one person's daily free-form text can learn their specific behavioural language patterns over time. The system operationalises a narrow but well-founded hypothesis: that the gap between a person's self-prediction and their actual behaviour is consistent enough, and linguistically detectable enough, to be modelled at the individual level.

The key technical contributions are: a blended inverse document frequency formula that resolves the cold-start problem from day one; a training protocol that updates the model once per daily message per week rather than once per week; L2 regularisation applied to the weight vector only, preserving the bias as a learnable base rate; and a fully on-device architecture with no server component and no data transmission.

The minimum viable product is live. The system works as designed.

6.2 Future Work

1. **Longitudinal evaluation.** Run the system over twelve or more weeks with real users and report MAE comparisons between model and self-prediction. This is the study the hypothesis requires.
2. **Dynamic vocabulary.** Let the model surface user-specific signal words from message history and propose them for addition to the relevant cluster. This extends the fixed vocabulary toward personalised feature discovery.
3. **Temporal decay.** Weight recent messages more heavily. Behavioural patterns shift over months; old labels should carry less influence on current predictions.
4. **Confidence display.** Surface $\hat{p}$ alongside the binary prediction, framed in plain language. Per Hengstler et al. [6], this is the correct path to adaptive trust calibration.
5. **Multi-user study.** Run FRISBEE across a cohort. Individual models stay private. Aggregate MAE comparisons would constitute statistically meaningful evidence for or against the central hypothesis.
6. **Cognitive load signal.** Use message length and vocabulary diversity as a proxy for cognitive load, per Sweller [1]. A secondary feature may improve prediction accuracy on weeks where emotional state, not language pattern, drives behaviour.

REFERENCES

[1] Sweller, John, "Cognitive Load During Problem Solving: Effects on Learning", *Cognitive Science*, 1988, 12, pp. 257–285.

[2] Kahneman, Daniel, *Thinking, Fast and Slow*, Farrar, Straus and Giroux, New York, 2011.

[3] Rebetez, Marie M. L. and Rochat, Lucien and Van der Linden, Martial, "Cognitive, Emotional, and Motivational Factors Related to Procrastination: A Cluster Analytic Approach", *Personality and Individual Differences*, 2015, 76, pp. 1–6.

[4] Anderson, Christopher J., "The Psychology of Doing Nothing: Forms of Decision Avoidance Result from Reason and Emotion", *Psychological Bulletin*, 2003, 129, pp. 139–167.

[5] Lee, John D. and See, Katrina A., "Trust in Automation: Designing for Appropriate Reliance", *Human Factors*, 2004, 46, pp. 50–80.

[6] Hengstler, Monika and Enkel, Ellen and Duelli, Selina, "Applied Artificial Intelligence and Trust: The Case of Autonomous Vehicles and Medical Assistance Devices", *Technological Forecasting and Social Change*, 2016, 105, pp. 105–120.

[7] Sparck Jones, Karen, "A Statistical Interpretation of Term Specificity and Its Application in Retrieval", *Journal of Documentation*, 1972, 28, pp. 11–21.

[8] Salton, Gerard and Buckley, Christopher, "Term-Weighting Approaches in Automatic Text Retrieval", *Information Processing and Management*, 1988, 24, pp. 513–523.

[9] Sharot, Tali, "The Optimism Bias", *Current Biology*, 2011, 21, pp. R941–R945.

[10] Dunning, David and Heath, Chip and Suls, Jerry M., "Flawed Self-Assessment: Implications for Health, Education, and the Workplace", *Psychological Science in the Public Interest*, 2004, 5, pp. 69–106.